Complete Java Classes

by Kunal Sir

Encapsulation (OOP's) in Java:

1. Encapsulation is one of the major pillars of OOP's concept.

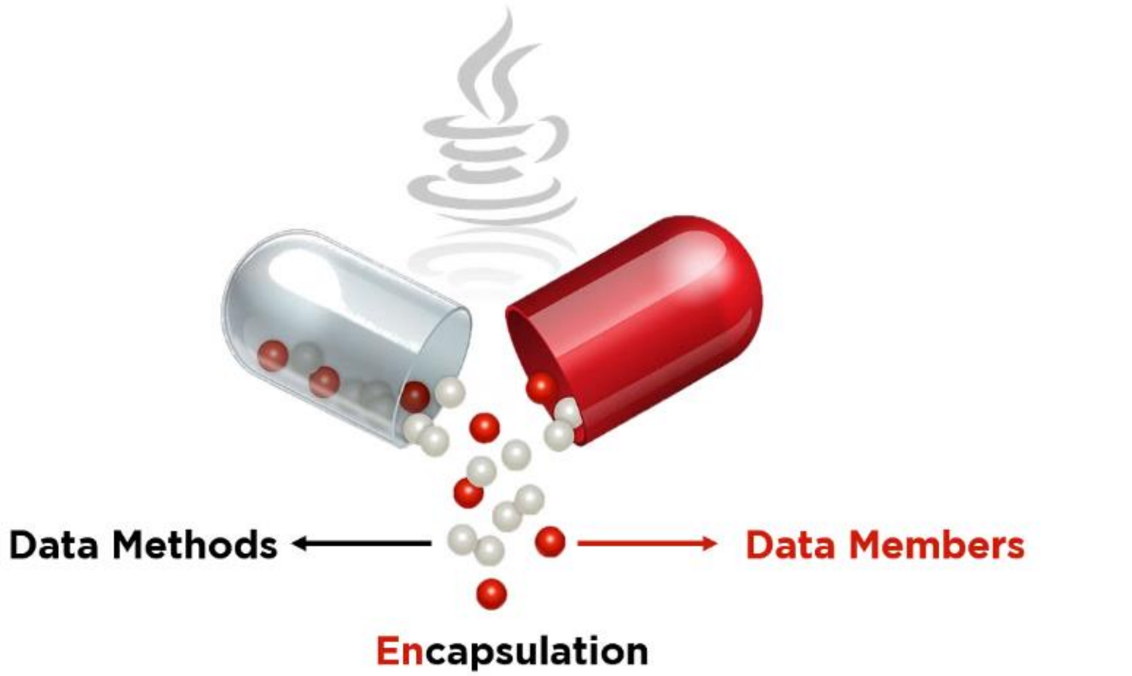
2. Wrapping of methods and variable in class is called as Encapsulation.

3. Encapsulation refers to the bundling of fields and methods inside a single class.

4. The purpose of encapsulation is to achieve Data Hiding.

5. Setter and getter methods are the perfect example of encapsulation.

6. We can achieve encapsulation through some access modifier like Private, Default, Protected and Public etc.



Encapsulation Diagram

What is Data Hiding?

- Data hiding is a way of restricting the access of our data members by hiding the implementation details.
- Encapsulation also provides a way for data hiding.
- We can use access modifiers to achieve data hiding.

by Kunal Sir

How Can We Achieve Encapsulation?

There are some access modifiers is provided by java to these access modifiers we can achieve encapsulation in java-

Private → In java program we can write like "private".

Default → In java program we can write like "default".

Protected → In java program we can write like "protected".

Public → In java program we can write like "public".

Scope of These Access Modifiers:

About Access Modifiers: - Access Modifiers are keywords that can be used to control the visibility of fields, methods, variables and constructors in a class. The four access modifiers in java are private, default, protected and public.

Scope of these access modifiers is given below –

Private: - As the name suggests, the scope of this modifiers is very restricted. When the data members and the class methods are prefixed with a private modifier, they can be accessed only in the class itself even the other classes of the same package cannot access the private members or methods.

For example: - Let's take an example of a Facebook Account when you change the privacy of your status to only for me, only you will be able to access the status, and no one even your friends will not be able to see your status.

Default: - When the instance members, methods of the class are not specified with any modifier. It is said to be having a default modifier by default. The instance variables and methods with default modifiers can be accessed only within the same package. Hence it is sometimes called package-private.

For example: - Let's take an example of a Facebook Account when you set your privacy status to "visible to your friends only". So you and yours's known friends can access your status.

Any other person will not be able to access your status information.

Protected: - We can use protected modifier methods, instance variable within the same package and access them in another package with the help of a child class which means we have to extend the class that contains protected variables in another package.

We cannot access them directly by creating the object of a class that contains protected members and functions.

Public: - The public modifier provides no restriction on the methods, classes and instance variable of the particular class.

When they are prefixed by the public modifier, they can be accessed in any package, in any class. The scope of the public modifier is very flexible as compared to the other modifiers.

For example: - Let's take an example of Facebook. When you set the privacy of status to the public everyone of facebook can access your status whether it is your friend, friend of your connection or not a friend.

Concise Information About Access Modifier:

Information is given in below table-

Access Modifier	Same class	Same package subclass	Same package non-subclass	Difference Package subclass	Different package non-subclass
Default	Yes	Yes	Yes	No	No
Private	Yes	No	No	No	No
Protected	Yes	Yes	Yes	Yes	No
Public	Yes	Yes	Yes	Yes	Yes

Understanding Program:

```
package com.cjc.karvenagar;
public class A
{
    protected int x;
}
```

```
package com.cjc.karvenagar;
public class B
{
      A a=new A( );
      public void m1( )
      {
            a.x=10;
      }
  }

package com.cjc.akurdi;
import com.cjc.karvenagar.*;
public class C extends A
{
     A a=new A( );
     public void m4( )
    {
        System.out.println(a.x); // incorrect
        System.out.println(x);

        C c=new C();
        System.out,println(c.x);
```

```
    }
 }

package com.cjc.akurdi;
public class D extends C
{
    public void m4( )
    {
       C c=new C( );
       System.out.println(c.x); // incorrect
       System.out.println(x);
       D d=new D( );
       System.out,println(d.x);
     }
 }
```

What is Tightly Encapsulated Class?

When each variable is declared private in a particular class, it is commonly termed a "**Tightly Encapsulated Class**" in java